Artificial Intelligence Usage

Primary Assistant: VS Code Copilot / GitHub Copilot

Models: GPT-5.3-Codex and Gemini 3.1 Pro

I treat LLMs as high-velocity execution partners. My approach centers on strict prompt engineering, strict constraints, and rigorous manual code review. I never treat AI output as production-ready without comprehensive validation.

The prompts I provide are detailed and strict. I dot not let the AI make architectural decisions, I provided highly specific, constrained prompts to maintain full design control.

I always check that the AI provides the optimal solution, not just a working one. If the solution is not the best one, I either rewrite it manually or re-prompt the AI with detailed instructions regarding the optimal solution.

Every line of generated code was manually reviewed, refactored, and optimized.

Examples

The examples below are prompts that I used to quickly generate working methods and initial logic flows for testing, which were subsequently deeply refactored, altered and optimized for the final release.

Example 1:

1. create helper function "ai_task" (args: prompt, user_message). The function sends to OpenAI the following messages:
[['role' => 'system', 'content' => 'You are a data extraction and text analysis engine. Do not include explanations. Do not infer missing or fabricate information. Do no add any additional text or comments. Use empty string for missing fields.'],['role' => 'user', 'content' => prompt . 'USER_MESSAGE: """ . user_message . """.]];
return only the text response.
2. If an escalation is pending (is_human_escalation_request !== False) :
- call "ai_task" function, prompt="If the USER_MESSAGE is a positive affirmation, confirmation, or approval (Examples: ok, confirmed, yes please, sure), respond exactly and only "YES". If it does not, respond exactly and only "NO". Always respond exactly "YES" or "NO" with no additional text, user_message= {current_user_message}"
- use the response to confirm or not human escalation

Example 2:

add new function "human_escalation_request"(args: reason) . The function must:

- check if variable "is_human_takeover"==true and if true, send to the user the message "A human agent has been contacted alrady, it will reply soon! and return.
- set a global context variable called "is_human_escalation_request" to reason arg. value
- send to user the message "Do you want to contact a human agent?" .
- add logic that trigger for every user message, it checks if is_human_escalation_request !== false, and if !== false, check if the current user message is a confirmation (examples: yes, ok, confirmed, yes please), or a cancellation (no thanks, cancel, no). if is confirmation call "human_escalation" function passing is_human_escalation_request, current user message, can_add_luggage as arguments. If cancellation send to user the messaage "Ok cancelled, what's next?". In all cases, if is_human_escalation_request !== false, set is_human_escalation_request = false

The "human_escalation_request" function must be sent in the following cases:

- User explicitly request to contact a human (create always active tool for this, desc=I want to contact a human support agent or team member. I want human support. How can I have support?). reason="explicit_user_request"
- The user want to add a baggage but "can_add_luggage"=false. reason="cannot_add_luggage"
- system error (within Exception cases). reason="system_error_{function_name}"

Add new function "human_escalation" (arg: booking_reference, reason, user_message). The function must:
- set global variable "is_human_takeover"=true
- send to the user the message "I contacted a human agent, you will get a reply soon!"

Help understanding how the luggage system works

The luggage-options API returns several details, and the add-luggage API body requires several details too. Because of this, I suspected that the system was more complex than a simple "add baggage A to flight B." I asked the AI to clarify, it helped me to understand how the system works.

Example 3:

analyze response_luggage_options.json and response_lookup.json file and tell me the logic for adding luggages, check if flight segments are involved, and how to manage luggages per passengers

Add log_event to all exceptions

Example 4: *Analyze main.py, tools.py, api.py, utils.py and use log_event() function to add log for every critical error (Exceptions)*